

CAPSTONE PROJECT REPORT

On

**Locus: A Gamified Mobile Learning Platform with
Asynchronous League-based Engagement for Competitive
Examination Practice**

Submitted to

LOVELY PROFESSIONAL UNIVERSITY

In partial fulfilment of the requirements for the award of degree of

Bachelor of Computer Applications

Submitted By

Vinay Kumar Tiwari (12324444)

Jhanvi Sharma (12303149)

Tanushka Raghuwanshi (12321549)

Nidhi (12324689)

G. Rajeshwari (12322513)

Supervised By

[Supervisor Name] ([ID])

Assistant Professor

Department of Computer Applications

Lovely Professional University, Phagwara, Punjab-144411, India

Project Group: [Project Group]

Section: [Section]

MAY 2026

CERTIFICATE

This is to certify that the project titled *"Locus: A Gamified Mobile Learning Platform with Asynchronous League-based Engagement for Competitive Examination Practice"* submitted by Vinay Kumar Tiwari (12324444), Jhanvi Sharma (12303149), Tanushka Raghuwanshi (12321549), Nidhi (12324689) and G. Rajeshwari (12322513), under the supervision of [Supervisor Name], Assistant Professor, School of Computer Applications, Lovely Professional University, is a record of original and bonafide work carried out by them for the partial fulfilment of the requirements for the award of the degree Bachelor of Computer Applications. The results embodied in this report have not been submitted to any other subject or university.

Supervisor Name: [Supervisor Name] ([ID])

Designation: Assistant Professor, SCA

Supervisor Signature:

Date:

DECLARATION BY STUDENT

We, Vinay Kumar Tiwari (12324444), Jhanvi Sharma (12303149), Tanushka Raghuwanshi (12321549), Nidhi (12324689) and G. Rajeshwari (12322513), hereby declare that the work done by us on "*Locus: A Gamified Mobile Learning Platform with Asynchronous League-based Engagement for Competitive Examination Practice*" from January 2026 to May 2026 under the supervision of [Supervisor Name], Assistant Professor, School of Computer Applications, Lovely Professional University, Phagwara, Punjab, is a record of original work for the partial fulfilment of the requirements for the award of the degree Bachelor of Computer Applications. The results embodied in this report have not been submitted to any other subject or university.

Vinay Kumar Tiwari (12324444)

Jhanvi Sharma (12303149)

Tanushka Raghuwanshi (12321549)

Nidhi (12324689)

G. Rajeshwari (12322513)

ACKNOWLEDGEMENT

First and foremost, we would like to thank God for the blessings that allowed us to complete this capstone project on time and with success. We would also like to express our thanks to the Dean and Head of School, School of Computer Applications, and to the Head of Department for the institutional support that made this project possible.

We would like to record our sincerest gratitude to our project supervisor, [Supervisor Name], Assistant Professor, School of Computer Applications, for the continuous support and patient guidance offered throughout the project work. The willingness to question our assumptions early — rather than at the end — shaped this project in ways that are not visible in the code, but are visible in how the product behaves.

We are equally thankful to the faculty members of the Department of Computer Applications for arranging review meetings, providing computing resources and creating an environment in which independent project work was possible. Our thanks also go to the lab staff who helped with installations and small but irritating issues that come up when one tries to deploy a Node.js stack on a shared machine, and to the peers and seniors who tested early builds of the application and reported the awkward parts that we had stopped noticing.

Finally, we would like to thank our families and friends for their support and patience during the long working sessions that a project of this size demands.

PUBLICATION DETAILS

PLAGIARISM DETAILS

CONTENTS OF THE REPORT

Contents	Page No
CHAPTER 1: INTRODUCTION	1–2
1.1 Report Organization	
CHAPTER 2: LITERATURE REVIEW	3–7
2.1 Mobile Learning and Bite-sized Practice	
2.2 Game-based Learning and Gamification	
2.3 Active Recall and Spaced Repetition	
2.4 League Design and Competitive Comparison	
2.5 Survey of Existing Indian Platforms	
2.6 Research Gaps Addressed	
CHAPTER 3: RATIONALE AND SCOPE OF THE STUDY	8–9
3.1 Rationale	
3.2 Problem Statement	
3.3 Scope of the Study	
CHAPTER 4: OBJECTIVES AND HYPOTHESIS OF THE STUDY	10–11
4.1 Objectives	
4.2 Research Hypotheses	
CHAPTER 5: RESEARCH METHODOLOGY	12–16
5.1 Architecture Design Phase	
5.2 Backend Module Construction	
5.3 Mobile Client and Admin Console Phase	
5.4 Pilot and Evaluation Phase	
5.5 System Flowchart — Exercise Completion Path	
5.6 Logical Architecture	
CHAPTER 6: COMPLETE WORK PLAN WITH TIMELINES	17
CHAPTER 7: EXPECTED OUTCOMES OF THE STUDY	18–19
7.1 Technical Deliverables	
7.2 Empirical Outcomes	

7.3 Practical Contributions

CHAPTER 8: RESEARCH AND EXPERIMENTAL WORK DONE **20–29**

.....

- 8.1 Backend Architecture and Module Layout
- 8.2 Hybrid Relational–Document Data Model
- 8.3 Gamification Subsystem
- 8.4 Hearts and Streak Utilities
- 8.5 The Asynchronous League and Leaderboard
- 8.6 Why Redis Sorted Sets for Live Ranking
- 8.7 The Mobile Client
- 8.8 The Admin Console
- 8.9 Authentication and Security
- 8.10 The Daily Worker

CHAPTER 9: RESULTS AND DISCUSSION **30–33**

- 9.1 The Pilot Setup
- 9.2 Quantitative Findings
- 9.3 Qualitative Findings
- 9.4 Performance Measurements
- 9.5 Bugs Worth Recording

CHAPTER 10: CONCLUSION AND SUMMARY **34–36**

- 10.1 Summary of Key Results
- 10.2 Limitations
- 10.3 Future Work
- 10.4 Closing Reflection

CHAPTER 11: REFERENCES **37–39**

CHAPTER 1. INTRODUCTION

Preparation for competitive examinations in India — JEE for engineering admissions, NEET for medical admissions, NIMCET for the Master of Computer Applications programme, and a long list of state and central tests — has moved decisively onto the phone. A school student in a tier-three town now has, in their pocket, a device that can host the same lectures, the same question banks, and the same mock tests that were once only available through expensive coaching centres. The catalogue of available content has exploded; the price of access has fallen sharply; and yet the most common complaint we hear from students preparing for these examinations is not about content. It is about the difficulty of staying consistent.

Most digital preparation platforms in this market have inherited the shape of their offline ancestors. They are built around long lecture videos and large pools of practice questions. The student is expected to watch, take notes, and then practise on their own time. This format is not wrong — it works for the disciplined learner who already has a study schedule and the willpower to follow it. The format is silent, however, on the much larger group of learners whose biggest obstacle is not understanding the material but turning up day after day. There is no streak that breaks when they skip a session, no group of twelve peers visibly racing them on a weekly leaderboard, no small daily reward that whispers *come back tomorrow*.

Outside the world of test preparation, the language-learning application Duolingo has shown — over hundreds of millions of users — that this missing layer matters. The pedagogy of language apps can be argued about endlessly, but their engagement design is not in question. A five-minute daily session is enough to keep a learner active. The streak counter, the points bar, the friendly mascot, the league of twelve players of similar level: each piece is small in isolation, and powerful in combination. The hypothesis that motivated this project is straightforward. Take the engagement loop that has worked for language learning, and apply it carefully to multiple-choice question (MCQ) practice for competitive examinations. Replace the sentence translation exercise with a Physics MCQ. Replace the matching exercise with a "match the formula to the law" exercise. Keep the streak. Keep the experience-points bar. Keep the leaderboard, but redesign it so that no learner is unfairly placed against peers who have been competing for a week longer than they have. This last point — the asynchronous, lobby-based leaderboard — is the most distinctive piece of the design we present in this report.

The system we built is called **Locus**. The name is deliberate. In Latin, *locus* means a place: in our framing, the place a student returns to every day for a few minutes of focused practice. The platform consists of three coordinated codebases — a NestJS back end, a React Native (Expo) mobile client and a Next.js admin console — sharing a PostgreSQL database and a Redis instance. The mobile client is the surface our learners see; the admin console is the surface our content team uses; the back end is where the entire reward economy is calculated, every time, inside a database transaction.

This report describes the design, the implementation and the small pilot evaluation of Locus. It is the outcome of one academic year of sustained engineering work, built around a problem that all five members of the team have lived through as students themselves.

1.1 Report Organization

Chapter 2 surveys the academic and industry literature in mobile learning, gamification, spaced repetition and competitive leaderboard design, and identifies the gaps the present work attempts to fill. Chapter 3 sets out the rationale for the project, the precise problem statement, and the boundaries of the study. Chapter 4 lists the objectives and the corresponding research hypotheses. Chapter 5 explains the research methodology, the system flowchart for the exercise-completion path and the logical architecture of the platform. Chapter 6 presents the work plan and timelines that were followed during the academic year. Chapter 7 describes the expected outcomes against which the project was assessed. Chapter 8 is the longest chapter — it documents the actual research and experimental work done, including the data model, the gamification subsystem, and the asynchronous league implementation. Chapter 9 reports on results from a small pilot study and discusses what they mean. Chapter 10 closes with a summary of the key findings and a roadmap for future work. Chapter 11 lists the references.

CHAPTER 2. LITERATURE REVIEW

2.1 Mobile Learning and Bite-sized Practice

The shift from desktop-centred e-learning to mobile-first learning has been one of the defining trends of the last decade. Researchers writing under the umbrella term *m-learning* have repeatedly noted that mobile learning is not simply desktop e-learning on a smaller screen. The constraints — small display, intermittent attention, frequent interruption, touch input — push designers toward micro-interactions: short, self-contained tasks that can be completed in a few minutes. Park in 2011 framed mobile learning as a fundamentally different pedagogical mode in which informal, individual and social learning each have a distinct place; this distinction has become more visible as products specifically designed for short attention windows have outperformed direct ports of desktop learning material.

For competitive examination preparation, this shift has been only partially absorbed. Many widely used platforms still expect the user to watch a lecture of forty-five to sixty minutes. The few that emphasise practice tend to copy the structure of paper-based question banks: long quizzes with no immediate feedback, no progression curve and no sense of momentum. There is a clear gap, in the Indian market in particular, for a product whose unit of interaction is the single MCQ rather than the single lecture, and whose pace is set by the student rather than by the content author.

2.2 Game-based Learning and Gamification

The literature draws a useful distinction between *game-based learning*, in which the learning content is delivered through an actual game, and *gamification*, in which game-like elements are added to a non-game activity. Locus belongs to the second category. The foundational text by Deterding and colleagues in 2011 defined gamification as the use of game design elements in non-game contexts. The systematic review by Hamari, Koivisto and Sarsa in 2014 surveyed empirical studies and concluded, with appropriate caveats, that gamification can produce positive effects on motivation and engagement, particularly when the rewards are meaningful and the social context is present.

The Self-Determination Theory work of Deci and Ryan — first published in 1985 and refined in many later papers — supplies the deeper explanation for why gamification works in some contexts and fails in others. External rewards can either support or undermine intrinsic interest, depending on whether they are perceived as informational (showing the learner that they are progressing) or controlling (telling the learner what to do). For exam preparation, the learner already has a strong external motivation in the form of the upcoming examination. Gamification here does not need to manufacture motivation from nothing; its job is to lower the daily

activation energy required to start a session, and to provide micro-feedback along the way that confirms the session was worthwhile.

Two large products demonstrate that this approach scales. Duolingo has popularised the streak, the league and the heart system. Khan Academy uses XP and badges with a lighter touch. Both have published talks describing the magnitude of the retention improvements they attribute to these mechanics. The exact numbers are commercially sensitive but consistently large.

2.3 Active Recall and Spaced Repetition

Two findings from cognitive psychology shape the long-term direction of Locus. The first, drawing on the line of work continued by Karpicke and Roediger in 2008, is the *testing effect*: retrieval practice produces stronger long-term memory than passive review. MCQ practice is itself a form of retrieval practice, which means that simply by making practice the central activity of the platform, Locus is aligned with this finding. The second is *spaced repetition* — the observation, going back to Ebbinghaus, that recall is best when material is reviewed at increasing intervals over time, rather than crammed. The SM-2 algorithm popularised by Wozniak in his work on SuperMemo, and later carried into tools like Anki, gives a concrete schedule for these intervals.

The current build of Locus does not implement true spaced repetition. Exercises are delivered in the order chosen by the content author. A planned future module — discussed in Chapter 10 — is a Revision Queue that schedules previously failed questions according to a simplified spacing algorithm. The hooks for this exist in the data model already.

2.4 League Design and Competitive Comparison

Leaderboards are the most-studied gamification element and also the most controversial. The work surveyed by Bai and colleagues in 2020 suggests that leaderboards motivate learners in the middle of the distribution most reliably, while learners at the very top and the very bottom are less moved by them. Older, ladder-style leaderboards in which every player competes against every other player tend to demoralise the weak and bore the strong.

The asynchronous, lobby-based design used in modern Duolingo is a deliberate response to this problem. Players are placed into fixed-size groups (twelve in Duolingo, twelve in Locus) and are promoted or demoted based on their relative ranking within that small cohort. Because the lobby starts only when it is full, every player gets the same season length. Because the cohort is small, the difference between the bottom and top of the lobby is psychologically tractable. The design is discussed in detail in Chapter 8.

2.5 Survey of Existing Indian Platforms

We surveyed the major Indian preparation platforms before settling on the design of Locus. A short, opinionated summary follows.

Unacademy, Byju's and Vedantu have very strong content libraries, particularly for JEE and NEET. Their emphasis is on recorded and live video. Practice tests exist but feel bolted onto the video catalogue. Engagement loops are minimal. **Testbook** and **Adda247** are test-series oriented and stronger for SSC, banking and railway examinations. Their practice flow is closer to what we wanted, but gamification is limited to a basic point system. The international benchmark for engagement design remains **Duolingo**, although its content is languages rather than examinations. **Khan Academy** is excellent in mathematics but is more web-first than mobile-first.

The conclusion we drew was that there is room for a product combining the bite-sized, gamified flow of a language application with the subject coverage and exam orientation of an Indian preparation platform. Locus is positioned in that gap.

2.6 Research Gaps Addressed

The literature surveyed above reveals a consistent pattern. Engagement mechanics are studied in isolation, usually within language-learning applications, and rarely in the specific context of multiple-choice preparation for high-stakes Indian competitive examinations. The architecture required to support a gamified preparation platform on commodity infrastructure — including a leaderboard that scales to many concurrent groups without becoming unfair — has received almost no systematic attention in the academic literature. Locus is designed to fill this gap. It contributes both an end-to-end architecture for a gamified MCQ practice platform and a specific design for an asynchronous league system that preserves the fairness of the season window even as new learners join the platform at arbitrary times.

CHAPTER 3. RATIONALE AND SCOPE OF THE STUDY

3.1 Rationale

The motivation for Locus came from observation rather than from a gap in the academic literature. Members of the team have, over the last three years, watched friends and juniors install one preparation application after another, use it intensively for a few days, and then quietly stop. The pattern was so consistent that it began to look like a property of the format itself rather than a property of any particular student. Long lecture videos demand a level of focused attention that competes with everything else on the phone — messages, short videos, notifications. Without an engagement layer that pulls the learner back daily, the app sits unused on the home screen until it is eventually deleted to free up storage.

What changed our thinking was a small experiment. Two members of the team kept their Duolingo streaks alive for more than two hundred days each, while admitting freely that they were no longer learning much French; the streak itself was the reason they opened the application. If a streak alone could pull a learner back day after day for a language they had stopped seriously studying, what could the same mechanic do for a subject the learner actually needed to clear an examination in? The premise of Locus is that the answer is "a great deal", provided the mechanic is implemented honestly.

The decision to build a complete platform — back end, mobile client and admin console — rather than only a prototype of the gamification layer was deliberate. Engagement loops are not features that can be evaluated in isolation. A streak counter that lives inside a stub application is not the same artefact as a streak counter inside a real platform with real onboarding, real content, real friends on the leaderboard and real Diamonds in the wallet. Only by embedding the mechanics inside a complete system could the team examine how they behaved in actual use.

3.2 Problem Statement

The challenge addressed by this work has three dimensions that have to be solved together rather than separately:

- **Engagement.** How can a mobile preparation platform hold a learner's daily attention for ten to fifteen minutes over the course of weeks and months without depending on push-notification spam or guilt-based reminders?
- **Fair competition.** How can a leaderboard system present a meaningful weekly competition to learners who join the platform at arbitrary times, without forcing all learners onto a common, globally synchronised season that disadvantages late joiners?
- **Operational simplicity.** How can the entire reward economy — experience points, in-app currency, lives, streaks and league standings — be calculated correctly under

concurrency, without the small but corrosive bugs (lost rewards, double spends, stuck seasons) that erode trust in a gamified system?

These three dimensions interact. Engagement depends on a leaderboard that feels fair. A leaderboard that feels fair depends on accurate season XP that, in turn, depends on a back end that calculates rewards atomically. Locus is designed to address all three together rather than in isolation.

3.3 Scope of the Study

The scope of this capstone project is a working, demonstrable system that supports the complete learner journey: registration, an onboarding flow that branches between exam-targeted and subject-curious learners, a catalogue of subjects organised into units and exercises, an exercise player that supports five distinct question formats, the full gamification subsystem, the asynchronous league leaderboard, an admin console for content authors and a small set of analytics dashboards. The scope explicitly does not include push notifications, payment integration for purchasing in-app currency, true offline mode, or formal accessibility audit, although hooks for each of these are documented in Chapter 10 as future work. The pilot evaluation in Chapter 9 is qualitative and small in scale; it is not an attempt at a controlled study.

CHAPTER 4. OBJECTIVES AND HYPOTHESIS OF THE STUDY

4.1 Objectives

The objectives of the project, agreed with the supervisor at the start of the academic year, are the following.

- **O1.** Design and build a mobile-first learning application supporting MCQ-style practice across multiple subjects and units, with a clean separation between catalogue content and learner state.
- **O2.** Implement a gamification subsystem comprising experience points (XP), an in-app currency named Diamonds, a hearts-based lives system and a daily streak counter, with all reward calculations performed inside database transactions so that no reward is ever partially applied.
- **O3.** Implement an asynchronous, league-based leaderboard such that learners are matched into fixed-size lobbies of twelve and competition begins only when the lobby fills, removing the unfairness of late joiners that characterises a global season design.
- **O4.** Build a backend service exposing a clean REST API, organised into domain modules (Auth, Courses, Gamification, Leaderboard, Admin, Jobs), and supported by a daily background worker that resets streaks for inactive users and closes expired leaderboard groups.
- **O5.** Build a Next.js administrator console for content management and operational monitoring, with a live preview of each question that mirrors the mobile rendering.
- **O6.** Use a hybrid relational–document database model — strict relational tables for the educational hierarchy and a flexible JSON column for question payloads — so that the system can support new question types without schema migrations.
- **O7.** Demonstrate the system through a small pilot with student users, gather feedback, and document findings, limitations and a future roadmap.

4.2 Research Hypotheses

The objectives above are paired with a small number of hypotheses that can in principle be tested. We frame them informally; the pilot in Chapter 9 was not powered to confirm or refute any of them rigorously.

- **H1.** A mobile MCQ practice application that implements a daily streak counter, an XP bar and a small twelve-player leaderboard will produce higher week-over-week return rates than an equivalent application without these mechanics, on the same content and the same learner population.

- **H2.** Replacing a globally synchronised weekly leaderboard with an asynchronous, lobby-based design (where the season begins only when twelve learners are present) will reduce the proportion of learners who report feeling that their leaderboard position is unfair, particularly among learners who joined the platform recently.
- **H3.** A hybrid relational–document data model allows a small team to add new question formats without database migrations and without measurable impact on read latency for the catalogue.
- **H4.** Calculating the entire reward economy — XP, Diamonds, streaks and league season XP — inside a single database transaction prevents the class of partial-application bugs that otherwise occur under concurrency on the exercise-complete endpoint.

H3 and H4 are essentially engineering claims and were verified by construction during implementation. H1 and H2 require a controlled study that is beyond the time budget of a single capstone year; the pilot in Chapter 9 provides preliminary evidence that is consistent with both but not sufficient to settle either.

CHAPTER 5. RESEARCH METHODOLOGY

5.1 Architecture Design Phase

The first phase of the project, occupying roughly the first six weeks, was the architecture design phase. The team began by mapping the journey of a single learner through the system: from sign-up, through onboarding, into a first exercise, through reward calculation, into the leaderboard. Mapping this journey before any code was written exposed the points where the design had to make non-trivial choices: how to model the dual onboarding flow (exam-targeted versus subject-curious), how to keep the question schema flexible across question types, and how to design a leaderboard that did not penalise late joiners. Each of these decisions is documented in Chapter 8.

The technology stack was chosen during this phase. The team considered Flutter against React Native for the mobile layer; the existing TypeScript experience in the team tipped the choice towards React Native via the Expo workflow. Express was considered against NestJS for the back end; NestJS was chosen because the structural conventions it enforces pay off as soon as the codebase grows beyond a few thousand lines. PostgreSQL was selected over MongoDB because the educational hierarchy benefits from referential integrity, and because PostgreSQL's JSON column support gives the schema flexibility required for question payloads without giving up the rest of the relational model. Redis was selected for the leaderboard because its Sorted Set primitive is exactly the right data structure for ranked membership.

5.2 Backend Module Construction

The second phase developed each backend module in isolation behind unit tests. Modules were brought into the application context in roughly this order: Auth, Users, Courses, Gamification, Leaderboard, Admin and Jobs. The order is significant. Auth and Users are dependencies of every other module; Courses must exist before Gamification can award XP for completing an exercise; the Leaderboard depends on the Gamification service emitting XP events; the Admin module re-uses the Courses and Users data models; and the Jobs module depends on all of the above to perform its periodic maintenance.

Each module was developed in three steps. First, the data model was added to the Prisma schema and a migration generated. Second, the service class was implemented with unit tests using an in-memory PostgreSQL spun up in Docker. Third, the controller was added and verified against the running application using HTTP-level tests. Keeping this discipline is the single most important reason the project was completed within the academic year.

5.3 Mobile Client and Admin Console Phase

Once the backend modules stabilised, the team built the mobile client and the admin console in parallel, with one sub-team on each. The mobile client uses the Expo Router and a small set of

reusable components (cards, headers, the heart strip, the celebration screen) to keep the visual language consistent. The admin console uses Next.js with the App Router, server components for data fetching and Tailwind CSS for styling. The two clients share no source code but do share their understanding of the API through TypeScript types generated from the Prisma schema and the DTO classes.

5.4 Pilot and Evaluation Phase

The final phase ran the small two-week pilot described in Chapter 9. The pilot was deliberately scoped small — twenty-one learners drawn from our own college and a friend's coaching centre — so that the team could observe usage closely, hold short interviews with each participant and react quickly to feedback. It was not designed as a controlled study and the numbers reported in Chapter 9 should be read as descriptive rather than inferential.

5.5 System Flowchart — Exercise Completion Path

The most important request path in the system is the exercise-completion path. The mobile client posts the result of a completed exercise to `POST /exercise/complete`. The back end then opens a database transaction and proceeds through the following sequence:

1. Look up the exercise and its questions; compute the score percentage and a perfect-score flag.
2. Award base XP and, if the score is one hundred percent, the perfect-score bonus.
3. Update the learner's streak in their local time zone, applying the same-day, consecutive-day or streak-broken case as appropriate; award the seven-day-multiple bonus in Diamonds when applicable.
4. Check whether this completion finishes the parent unit; if so, award the unit-completion bonus in Diamonds and write a row in the Diamond ledger.
5. Insert an exercise-attempt row recording score, XP awarded, Diamonds awarded and the perfect flag.
6. If the learner is in an active leaderboard group, increment their season XP on the participant row.
7. Commit the transaction.
8. After commit, update the Redis sorted set for the leaderboard group so the live ranking reflects the new season XP. If this Redis write fails, the database is still consistent and the next read will repopulate Redis from the truth in PostgreSQL.

This flow is the single most carefully engineered piece of the back end. Three of the four bugs that consumed the most debugging time during the pilot lived inside it; each is documented in Chapter 9.

5.6 Logical Architecture

At the highest level, Locus is a three-tier system. The mobile client and the admin console are both clients of the back end. The back end owns all business logic and is the only component that talks to PostgreSQL or Redis. PostgreSQL is the source of truth for every piece of state; Redis is a cache and a fast read-side replica for the live leaderboard ranking.

Inside the back end, modules are organised by bounded context rather than by technical layer. Auth handles sign-up, sign-in and JWT verification. Users owns the profile and account-level operations. Courses owns the catalogue. Gamification owns the reward economy. Leaderboard owns lobby placement, season XP and promotion or demotion. Admin exposes content-management endpoints behind an admin guard. Jobs runs the daily worker. Prisma and Redis are wrapped as injectable services so that any module can use them through a constructor.

Deployment in the pilot is intentionally simple. A single virtual machine runs the backend container, the PostgreSQL container and the Redis container under `docker-compose`. The admin console runs as a separate Next.js application, fronted by Nginx. The mobile build is produced by Expo Application Services and side-loaded onto pilot devices. A small `deploy.sh` script handles `git pull`, the Prisma migration and the controlled restart. For production the database would move to a managed service and the back end would run behind a load balancer with two or three instances, but the architecture itself does not change.

CHAPTER 6. COMPLETE WORK PLAN WITH TIMELINES

The work plan agreed with the supervisor at the start of the academic year was structured as four overlapping phases. The actual schedule followed the plan closely; the only significant deviation was that the asynchronous league subsystem took three weeks longer than estimated, partly absorbed by reducing the time spent on push notifications and the offline mode (both deferred to future work, see Chapter 10).

Phase 1 — Architecture and Data Model (Weeks 1–6). Requirement gathering, stakeholder interviews, drafting of the hybrid relational–document schema, selection of the technology stack, preparation of the project repository with shared lint, format and type-check configuration, deployment of the development environment under Docker.

Phase 2 — Backend Module Construction (Weeks 5–14). Auth and Users first, then Courses, then Gamification, then Leaderboard, then Admin, then Jobs. Each module was developed with unit tests against an in-memory PostgreSQL instance and integration tests against the running application.

Phase 3 — Mobile Client and Admin Console (Weeks 10–22). Two sub-teams worked in parallel. The mobile sub-team produced the authentication, onboarding, home, subjects, exercise player, leaderboard, profile, shop and streak screens. The admin sub-team produced the catalogue, users and operations screens. Mid-phase demo to the supervisor in Week 16; mid-phase demo to a panel of peers in Week 18.

Phase 4 — Content Authoring, Pilot, Report (Weeks 20–28). Authoring of the seed content set across seven subjects. Pilot recruitment and onboarding in Week 24. Two-week pilot from Week 24 to Week 26. Analysis of pilot data, qualitative interviews and preparation of the final report from Week 26 to Week 28.

The four phases overlap deliberately. Beginning the mobile client in Week 10, while several backend modules were still under development, allowed the front-end team to surface API design problems early enough that they could be fixed without a costly migration. Beginning content authoring in Week 20, while the exercise player was still being polished, surfaced UI issues with specific question types (in particular the MATCH and REORDER formats) early enough that the player could be fixed before the pilot.

The Gantt-style summary that follows lists, for each major deliverable, the weeks in which it was active. A "+" marks the week in which the deliverable was demoed to the supervisor.

Deliverable	Weeks
Requirements and architecture	1–4

Database schema and migrations	3–6 +
Auth and Users modules	5–8
Courses module	7–11 +
Gamification subsystem	9–16 +
Asynchronous league subsystem	12–20 +
Daily background worker	16–18
Mobile client construction	10–22 +
Admin console construction	12–22
Content authoring (seed)	20–24
Pilot deployment	23
Pilot study with learners	24–26 +
Analysis and report writing	26–28 +

CHAPTER 7. EXPECTED OUTCOMES OF THE STUDY

7.1 Technical Deliverables

The technical deliverables expected at the close of the academic year were the following. The actual deliverables, summarised in Chapter 8 and Chapter 9, line up with this list.

- A NestJS backend exposing a clean REST API, with domain modules for Auth, Users, Courses, Gamification, Leaderboard, Admin and Jobs, supported by a daily background worker.
- A Prisma schema implementing the hybrid relational–document model, with one migration history under version control.
- A React Native mobile client built with Expo, supporting the full learner journey from sign-up through onboarding into daily practice, including the leaderboard and the in-app shop.
- A Next.js administrator console with a live preview of each question that mirrors the mobile rendering.
- A Docker Compose deployment that brings up the back end, PostgreSQL and Redis with one command.
- Seed content covering Physics, Chemistry, Biology, Mathematics, Computer Science fundamentals, Python and C++.
- A short script and a small README that allow a new developer to set up the entire stack on a fresh machine in under one hour.

7.2 Empirical Outcomes

The empirical outcomes expected from the small pilot study were qualitative rather than quantitative. The team aimed to answer several specific questions:

- Does the streak counter actually pull learners back to the application on days they would otherwise have skipped?
- Does the asynchronous, lobby-based leaderboard feel fair to learners who join the platform after their peers?
- Is the heart system perceived as a reasonable cost on careless answers, or is it perceived as a punishment?
- Is the dual onboarding flow (exam versus subject) clearer than a single-path onboarding would have been?

The pilot is reported in Chapter 9. The short version is that the streak and the dual onboarding flow worked as expected; the leaderboard worked but its motivational effect was concentrated in the middle of the ranking; and the heart system divided opinion sharply, suggesting a planned softening of the rules.

7.3 Practical Contributions

Beyond the technical deliverables and the empirical findings, the project is expected to contribute three practical artefacts that are useful beyond the immediate work. First, a documented architecture for a gamified mobile preparation platform on commodity infrastructure, suitable as a starting point for similar projects. Second, a worked design and implementation of an asynchronous, lobby-based leaderboard, including the placement algorithm, the season-end ranking algorithm and the daily worker that closes expired groups. Third, a small but working seed of content across seven subjects, organised in a way that other content authors can extend without a developer in the loop.

CHAPTER 8. RESEARCH AND EXPERIMENTAL WORK DONE

8.1 Backend Architecture and Module Layout

The Locus back end is a single NestJS application, organised into one module per bounded context. The directory layout mirrors the modules: `auth/`, `users/`, `courses/`, `gamification/`, `leaderboard/`, `admin/`, `jobs/`, with shared `prisma/` and `redis/` infrastructure modules that any other module can pull in through dependency injection.

Each domain module follows the same internal pattern. A `Service` class owns the business logic and is the unit of testing. A `Controller` class is thin and translates HTTP into service calls. DTO classes annotated with `class-validator` decorators define the shape of every write request; the global validation pipe rejects unknown fields and malformed types before any controller code runs. Because the controllers are thin, the meaningful tests live one layer down, where the same service can be exercised from many different paths.

This layout is deliberately conservative. Splitting the back end into separate microservices was discussed early on — for example, peeling off the leaderboard as a separate service. The team decided against it. The domain is bounded, the operational complexity of multiple services would have eaten time better spent on product features, and the strong module boundaries inside NestJS provide most of the encapsulation benefits of physical service splits without the deployment cost.

8.2 Hybrid Relational–Document Data Model

The data model follows a hybrid relational–document pattern. The educational hierarchy and the gamification state are modelled relationally, where joins are predictable and integrity matters; the actual content of a question is stored as a JSON-encoded payload in a single content column on the `Question` table. This means the schema does not need to change every time the team adds a new question type. New types are added by writing a renderer in the mobile client and a small validator on the back end.

The educational hierarchy is *Subject* → *Unit* → *Exercise* → *Question*. A subject like Physics exists at three levels — foundation, intermediate and advanced — represented as three separate `Subject` rows distinguished by a `level` field and a `displayName` such as *Physics 1*, *Physics 2* or *Physics 3*. We considered modelling the level as a separate dimension of a single Physics subject; doing so would force every screen and every API call to disambiguate. Treating each level as a distinct subject keeps the API simple and avoids accidentally mixing easy and hard content in one lesson tree.

Exams are first-class. The `Exam` table is joined to `Subject` through a many-to-many table `ExamSubject`, so a single subject can appear under multiple exams without duplication. The join row carries an `orderIndex` so that an exam can present its subjects in a chosen order.

Onboarding produces a row in either `UserExamEnrollment` (for learners who picked an exam target and an exam date) or `UserSubjectEnrollment` (for learners who picked a subject and a self-rated difficulty of one, two or three). The two tables are deliberately separate so that the constraints stay clean: an exam enrollment requires an exam date; a subject enrollment requires a difficulty integer.

8.3 Gamification Subsystem

The gamification subsystem owns the entire reward economy. Its constants live in `rewards.constants.ts` rather than in the database, so that tuning them is a developer-controlled action that goes through code review. The current values are summarised below.

Constant	Value	Notes
EXERCISE_BASE_XP	20	Awarded for each completed exercise.
PERFECT_SCORE_BONUS_XP	5	Added on a 100% score.
UNIT_COMPLETION_DIAMONDS	100	Awarded once per unit, the first time it is fully cleared.
STREAK_7_DIAMONDS	150	Awarded on every seventh day of an active streak (7, 14, 21 ...).
HEARTS.MAX	5	Maximum hearts.
HEARTS.REFILL_COST_DIAMONDS	10	Cost in Diamonds to refill one heart.
HEARTS.REGEN_INTERVAL_MS	1,800,000	One heart every thirty minutes.
LEAGUE.GROUP_SIZE	12	Players per leaderboard lobby.
LEAGUE.SEASON_DAYS	10	Days an active season runs.
LEAGUE.PROMOTION_TOP	5	Top five promote at season end.
LEAGUE.DEMOTION_BOTTOM	3	Bottom three demote (if not at minimum tier).

The most important method on the gamification service is `completeExercise`. It opens a transaction, computes the score and the perfect flag, awards XP and the perfect bonus, updates the streak using the user's local time zone, checks for unit completion (with its Diamond bonus), inserts the exercise-attempt row, writes any Diamond ledger entry, increments the leaderboard season XP if the learner is in an active group, and commits. Only after the commit does the service push the new season XP into the Redis sorted set for the live leaderboard ranking. The transactional-inside, eventually-consistent-outside pattern prevents the worst kind of bug in a

gamified system, which is the bug where the user is told they earned XP but the balance does not actually move.

8.4 Hearts and Streak Utilities

The hearts subsystem implements a soft difficulty mechanism. A wrong answer in an exercise consumes one heart; right answers do not change the count. Hearts regenerate at one heart every thirty minutes, and a learner can spend ten Diamonds to refill one heart on demand. Hearts are capped at five. Regeneration is computed lazily — there is no per-user timer. On every read of `UserStats` the service computes the number of hearts the learner should have based on `lastHeartRefillAt` and `now`, clamps to the maximum and persists any change. This makes the system stateless from the worker's point of view.

The streak subsystem encapsulates the small but tricky logic of deciding when a learner's streak should grow, stay or reset. The four cases are: the first ever activity sets the streak to one; an activity on the same local day as the previous activity does not change the streak; an activity exactly one local day after the previous activity increments the streak by one; and an activity that skips at least one local day resets the streak to one. A daily worker walks users for whom the local date has crossed into a new day and resets the streak to zero where activity was missed. The job is idempotent: running it twice does no harm because it only writes when the streak is currently above zero and the day was indeed missed. Time-zone correctness was the source of one of the more painful bugs in the pilot, documented in Chapter 9.

8.5 The Asynchronous League and Leaderboard

The leaderboard is the most architecturally interesting feature of Locus. A naive design — "everyone plays in the same season, the top players are promoted, repeat" — has two well-known problems: late joiners are at a disadvantage because they have less time to earn XP, and sufficiently large user bases make the leaderboard impersonal because the top spots are dominated by a small group of power users. We adopted an asynchronous lobby model inspired by similar designs in modern Duolingo and competitive mobile games.

A `LeaderboardGroup` represents a single twelve-player lobby in a specific league tier. It has a state of `WAITING`, `ACTIVE` or `CLOSED`. When a learner completes their first exercise after their previous group has closed, the placement query looks for any group in the learner's current league tier with state `WAITING`. If one exists with fewer than twelve members, the learner is added; otherwise, a brand-new group is created with this learner as its first member. While a group is `WAITING`, XP earned by its members does *not* count toward `seasonXp`. The mobile UI tells the learner explicitly that they are waiting and shows the current fill level (for example, *Waiting for opponents (5/12)*).

When the twelfth learner joins, the group transitions to `ACTIVE`, `startedAt` is set to the current time and `endsAt` is set to ten days later. From that exact moment, every XP increment to a member also bumps `seasonXp`. Internally this happens inside the same transaction as the XP award, which is critical for correctness. After the transaction commits, the Redis sorted set for that group is updated, so the live ranking is always accurate.

A daily worker sweeps for groups with state `ACTIVE` and `endsAt` < `now`. For each such group it ranks the participants by `seasonXp` (with the tie-break rule that earlier `lastXpAt` wins ties), applies the promotion, safe and demotion bands, and writes the new `currentLeagueIndex` on each user. The group is then transitioned to `CLOSED`. New activity by these users will cause them to be placed into a new `WAITING` group at their (possibly updated) tier on the next exercise complete.

Position in group at season end	Outcome
1 – 5 (top five)	Promoted to next tier (capped at 30)
6 – 9 (middle four)	Stay in current tier
10 – 12 (bottom three)	Demoted to previous tier (floored at 1)

The advantage of this design is that every learner's season is fair: it lasts exactly ten days, no matter when they joined. The cost is a more complex placement query and a small window during which learners feel they are "just waiting". In practice the waiting window is short — at our pilot scale a group typically filled within a few hours of being opened.

8.6 Why Redis Sorted Sets for Live Ranking

The leaderboard ranking is read frequently — every time a learner opens the leaderboard screen or finishes an exercise. Computing rank in PostgreSQL would be possible with a window function over the participants of a group, but every read would trigger a sort over up to twelve rows. That is fine at twelve rows and one group, but it scales poorly when there are many groups and many concurrent reads. Redis sorted sets give the team exactly the operations needed: `ZADD`, `ZINCRBY`, `ZREVRANK` and `ZREVRANGE WITHSCORES`. Each is logarithmic in the size of the set, and the keys are independent, which makes sharding by group straightforward if it is ever needed.

PostgreSQL remains the source of truth. If Redis is wiped, a small bootstrap routine reads `LeaderboardParticipant` rows and rebuilds the sorted sets. We tested this in development by deliberately flushing Redis in front of a running app, and the leaderboard recovered without any user-visible damage.

8.7 The Mobile Client

The mobile client is built with Expo. The interesting parts are the exercise player at `app/exercise/[id].tsx`, which renders different question types through a switch on `question.type`, and the home screen, which subscribes to a `UserStatsContext` to keep the XP, Diamonds, Hearts and streak counters in sync after a session. The home screen also surfaces the current league and the time remaining in the season for learners whose group is active.

A notable engineering decision was to keep the exercise answers in local state until the very end and to post them to the back end in one shot. We initially considered an "answer per tap" pattern with an immediate API call after every question. That would have given finer analytics, but it would also have made the experience fragile on flaky networks. Sending the whole result at the end is simpler, faster from the user's perspective, and only sacrifices the ability to record partial sessions — an acceptable trade.

8.8 The Admin Console

The admin console is built with Next.js using the App Router and Tailwind CSS. Pages live under `app/(console)/` and are protected by a middleware that checks for an admin session cookie. A small client-side API helper and a server-side admin API helper keep all the network code in one place. The most useful screen during the pilot turned out to be the question editor. It renders a JSON editor for the content field with a small live preview that mimics the mobile rendering. Several content authors caught their own mistakes in the preview before saving — a tiny feature that paid for itself many times over.

The console is not only a CRUD form generator. It also surfaces analytics: daily active users, exercise completion counts per subject and average score per exercise. These dashboards proved essential during the pilot for spotting questions that were either trivially easy or unintentionally impossible — the analytics turned up one question with a three-percent correct rate that was traced to a typo in the answer key.

8.9 Authentication and Security

Authentication uses email and password, with bcrypt hashing on the server (cost factor ten). The JWT carries the user identifier and an issued-at timestamp; we deliberately keep the payload small. Token verification uses Passport's JWT strategy and the `JwtAuthGuard` is attached at the controller level for all protected routes. A separate `AdminGuard` checks both the JWT and an `isAdmin` flag, which is set by an administrator manually in the current build. In production this flag would be promoted into a roles table.

Security measures in place include bcrypt for passwords, DTO-level validation for every write endpoint, parameterised queries through Prisma, environment-based secrets that are excluded from version control, CORS configured to allow only known origins, and a separate admin guard

for the administrative endpoints. A formal penetration test was outside the scope of the project; it is listed in the future work in Chapter 10.

8.10 The Daily Worker

A small worker, defined under `jobs/`, runs at one minute past midnight UTC. It performs three sweeps. First it closes any leaderboard group whose `endsAt` is in the past, ranks its participants and applies the promotion or demotion to each. Second, it walks users whose local date has crossed and whose `lastActivityDate` is older than yesterday, resetting their streak to zero where activity was missed. Third, it sweeps users whose hearts are below the maximum and whose `lastHeartRefillAt` is older than the regeneration interval, refilling them. Each sweep is idempotent and safe to run more than once.

CHAPTER 9. RESULTS AND DISCUSSION

9.1 The Pilot Setup

A two-week pilot was run with a small group of student users — twenty-one in total, drawn from our own college and from a friend's coaching centre. The pilot was not a controlled study. Its goal was to validate that the system works end-to-end under real use, and to gather qualitative feedback on the gamification mechanics. Pilot users were asked to install the APK on their phones, sign up, complete onboarding and then use the application at their own pace for two weeks. They were given no schedule and no quotas. We observed usage through the analytics screens in the admin console and held two short interviews with each participant — one at the start, one at the end.

9.2 Quantitative Findings

The numbers below are summary statistics from the pilot. They are not statistically powerful, given the small sample, but they are honest.

Metric	Value
Total users	21
Users still active in week 2	16 (76 %)
Median exercises per active user per day	3
Median session length	6 minutes
Total exercises completed	612
Average exercise score	71 %
Users who reached a 7-day streak	9
Users who broke a streak at least once	13
Users who refilled a heart with Diamonds	7
Leaderboard groups formed	4
Leaderboard groups completing a full season	2

The seventy-six percent week-two retention is encouraging, given that no notifications, no marketing and no external incentives were used during the pilot. The relatively short session length (six minutes median) is in line with the design assumption that this application should fit into small windows of time rather than replace dedicated study.

9.3 Qualitative Findings

The interviews surfaced four themes. Almost every active user mentioned the streak as a reason they opened the application on at least one day they would otherwise have skipped. The flip side is that two users who broke long streaks expressed real disappointment, suggesting that a streak-freeze mechanism is overdue.

The hearts mechanism divided opinion. Some users found it a fair cost on careless answers. Others found it frustrating, especially on hard questions where they felt the wrong answer was honestly tried. A planned change — "no heart loss on questions tagged as hard, the first time only" — has been added to the backlog as a result.

The leaderboard motivated mid-rankers most. Users who found themselves in positions five through seven in their group were the most likely to log in to defend or improve their rank. Users at the very top or the very bottom were less moved by the leaderboard. This pattern matches the literature reviewed in Chapter 2 and suggests that small additional rewards — for example, a one-off Diamond bonus for moving up a position in the final 24 hours of a season — could spread the motivation more broadly.

The dual onboarding choice between exam target and subject focus was reported as natural. Eleven of twenty-one learners chose the subject path, which was higher than we expected. The implication is that the platform should not be marketed only as an exam tool; "improve in Physics" is itself a goal for many learners.

9.4 Performance Measurements

API latency held up well during the pilot. On the pilot deployment, median response times for the most-called endpoints were as follows.

Endpoint	Median (ms)	p95 (ms)
GET /catalog/subjects	18	43
GET /exercise/:id	27	58
POST /exercise/complete	86	211
GET /leaderboard	14	31
GET /me/stats	11	22

The POST /exercise/complete endpoint is the heaviest because of the transaction it carries. The latency is acceptable; the ninety-fifth percentile of around two hundred milliseconds means the user experiences the celebration screen as instant.

A separate small load test using k6 simulated one thousand concurrent virtual users, each completing one exercise per minute for ten minutes. The back end held up well: median latency on the same endpoint stayed at around eighty milliseconds and the ninety-fifth percentile stayed under two hundred and fifty milliseconds. PostgreSQL and Redis CPU usage were both modest. We did not attempt to load test beyond this, partly because the single-VM pilot deployment would have hit its own infrastructure limits well before the application logic did.

9.5 Bugs Worth Recording

Three bugs from the pilot are worth recording because each taught a small lesson that is likely to be useful to anyone building a similar system. The first was a streak counter that was off by one on the time-zone boundary; the early implementation compared dates in UTC, and a learner studying late at night in IST broke their own streak. The fix was to compare local dates by passing the user's time zone into the streak comparison. The lesson is that dates are not numbers and must not be compared naively.

The second was a double-spend on the heart refill. A learner with exactly ten Diamonds tapped the refill button twice in quick succession on a slow network. Both requests reached the server, both checks passed, and one of them produced a negative balance. The fix was to wrap the refill in a transaction with a `WHERE diamonds >= cost` clause on the update, so the second request fails atomically. The lesson is that optimistic clients meet pessimistic servers.

The third was a stuck `WAITING` group. During a quiet period a half-full group sat in the `WAITING` state for two days, while new users were being assigned to a different waiting group in a different tier (because their tier had changed in the meantime). The first group simply never filled. The fix was to add a staleness check in the daily worker that closes any waiting group older than fourteen days and refunds its participants to fresh placements. The lesson is that state machines need an escape hatch.

CHAPTER 10. CONCLUSION AND SUMMARY

This report has presented Locus, a gamified mobile learning platform for competitive examination practice. The system was built as three coordinated codebases — a NestJS back end, a Next.js admin console and a React Native mobile client — sharing a PostgreSQL database and a Redis instance. The back end is organised by bounded context into modules for Auth, Users, Courses, Gamification, Leaderboard, Admin and Jobs. The data model uses a hybrid relational–document pattern: a strict relational hierarchy of Subject → Unit → Exercise on top of a flexible JSON column for question payloads, so that different question types can share one storage schema without forcing a migration each time a new format is introduced. Redis sorted sets power the live leaderboard ranking, giving logarithmic-time updates and instant rank lookups. Gamification logic — XP awards, streak updates, heart regeneration, league promotion and demotion — runs inside database transactions and is supported by a daily background worker.

The most distinctive piece of the design is the asynchronous league subsystem. Learners are placed into fixed-size lobbies of twelve in their current league tier. While the lobby is `WAITING`, XP does not count toward season standings; the moment the lobby fills, the season begins, the ten-day countdown starts, and every XP award also bumps season XP. At season end the top five promote, the middle four stay, and the bottom three demote. The advantage of this design over a globally synchronised leaderboard is that every learner's season is fair — it lasts exactly ten days, no matter when they joined. The pilot results presented in Chapter 9 are consistent with this advantage, although a controlled study would be needed to confirm it.

10.1 Summary of Key Results

The pilot, while small, produced four findings worth carrying forward. First, week-two retention was seventy-six percent without any notifications or marketing, which is a strong baseline for a gamified preparation platform. Second, the streak counter demonstrably pulled learners back on days they would otherwise have skipped, which is the central engagement claim of the design. Third, the asynchronous, lobby-based leaderboard worked as designed, with seasons starting and closing on schedule and rank updates reflected in real time on the mobile screens. Fourth, the heart system divided opinion sharply, suggesting that the rules need to be softened — particularly on hard questions, the first time they are attempted — and that a streak-freeze mechanism should be added before the platform is opened to a wider audience.

API latency stayed within target ranges throughout the pilot. The exercise-complete endpoint, which carries the entire reward transaction, had a ninety-fifth-percentile latency of around two hundred milliseconds. Read endpoints were typically under fifty milliseconds. A small load test confirmed the back end can handle one thousand concurrent virtual users on the pilot hardware without errors, which is well beyond the immediate need.

10.2 Limitations

The limitations are honest and worth listing. Content volume is small — the pilot covered roughly fifty units across seven subjects, and a real launch would need ten times that. There is no true spaced repetition; exercises are delivered in the order chosen by the author. There are no push notifications, which would make streak reminders and league updates far more effective. The application has no offline mode, which is a real disadvantage on flaky networks. The iOS build is via TestFlight only; submission to the App Store has its own process and was outside the project's time budget. Deployment is single-region. There is no formal accessibility audit. There is no payment integration for direct purchase of in-app currency. The administrator role model is a single binary flag rather than a granular role table.

10.3 Future Work

Several items are queued for the next phase of work. A spaced-repetition revision queue, drawing from the SM-2 algorithm and adapted for MCQ where item difficulty is influenced by both response time and historical correctness. Streak freezes and weekend amnesty, addressing the most common piece of negative feedback from the pilot. Push notifications via Expo's notification service. Offline mode through a client-side cache and an outbox for posting completion submissions. Adaptive difficulty using the rich exercise-attempt history to recommend the next exercise rather than relying on author order. A bulk-import tool for content authoring. A question-difficulty calibration screen. Granular admin roles. Multi-region deployment. A web client sharing UI components with the mobile application.

10.4 Closing Reflection

Working on Locus has taught the team a small number of lessons that no single course on its own would have. The first is that a working system is the sum of many small decisions, none of them heroic — the choice to put the heart logic in a utility file, the choice to compare dates in the user's local time zone, the choice to use Redis sorted sets for the leaderboard. Each of these removed a problem that would otherwise have grown into a much bigger one. The second is that gamification is design, not just code. The numbers in the rewards constants file shape the user's experience as much as anything in the UI, and there is no substitute for watching real learners to see what feels good and what feels punishing. The third, and the lesson the team will carry forward, is that the unglamorous parts of a system — the migrations, the validation, the transaction boundaries, the daily worker — are what decide whether the impressive parts ever get to be seen. We are grateful for the chance the capstone gave us to build all of these in a single project, end to end.

CHAPTER 11. REFERENCES

1. Bai, S., Hew, K. F., and Huang, B. "Does gamification improve student learning outcome? Evidence from a meta-analysis and synthesis of qualitative data in educational contexts." *Educational Research Review*, 30, 2020, 100322.
2. Deci, E. L., and Ryan, R. M. *Intrinsic Motivation and Self-Determination in Human Behavior*. Plenum Press, New York, 1985.
3. Deterding, S., Dixon, D., Khaled, R., and Nacke, L. "From game design elements to gamefulness: defining gamification." *Proceedings of the 15th International Academic MindTrek Conference*, 2011, pp. 9–15.
4. Hamari, J., Koivisto, J., and Sarsa, H. "Does gamification work? A literature review of empirical studies on gamification." *47th Hawaii International Conference on System Sciences*, 2014, pp. 3025–3034.
5. Karpicke, J. D., and Roediger, H. L. "The critical importance of retrieval for learning." *Science*, 319 (5865), 2008, pp. 966–968.
6. Park, Y. "A pedagogical framework for mobile learning: categorizing educational applications of mobile technologies into four types." *International Review of Research in Open and Distributed Learning*, 12 (2), 2011, pp. 78–102.
7. Pimsleur, P. "A memory schedule." *The Modern Language Journal*, 51 (2), 1967, pp. 73–75.
8. Sailer, M., Hense, J. U., Mayr, S. K., and Mandl, H. "How gamification motivates: an experimental study of the effects of specific game design elements on psychological need satisfaction." *Computers in Human Behavior*, 69, 2017, pp. 371–380.
9. Werbach, K., and Hunter, D. *For the Win: How Game Thinking Can Revolutionize Your Business*. Wharton Digital Press, 2012.
10. Wozniak, P. A. *Optimization of Learning: SuperMemo Method*. Doctoral thesis, University of Economics, Wroclaw, 1990.
11. Mozer, M. C., Pashler, H., Cepeda, N., Lindsey, R., and Vul, E. "Predicting the optimal spacing of study: a multiscale context model of memory." *Advances in Neural Information Processing Systems*, 2009.
12. Hwang, G. J., and Wu, P. H. "Advancements and trends in digital game-based learning research: a review of publications in selected journals from 2001 to 2010." *British Journal of Educational Technology*, 43 (1), 2012, E6–E10.

13. Chou, Y. *Actionable Gamification: Beyond Points, Badges, and Leaderboards*. Octalysis Media, 2015.
14. Owen, V. E., Roy, M. H., et al. "Player identification in educational games." *Proceedings of the International Conference on the Foundations of Digital Games*, 2016.
15. NestJS Documentation. <https://docs.nestjs.com> (accessed during project work).
16. Prisma Documentation. <https://www.prisma.io/docs> (accessed during project work).
17. PostgreSQL Documentation, Chapter on JSON Types. <https://www.postgresql.org/docs> (accessed during project work).
18. Redis Documentation, Sorted Sets. <https://redis.io/docs/data-types/sorted-sets> (accessed during project work).
19. React Native and Expo Documentation. <https://reactnative.dev> and <https://docs.expo.dev> (accessed during project work).
20. Next.js Documentation. <https://nextjs.org/docs> (accessed during project work).
21. OWASP Foundation. *OWASP Top Ten 2021*. <https://owasp.org/Top10> (accessed during project work).